



CCNP: Setting Up A LLM On A Local Device Accessible From The Local Network

Kazu Li-Hirata: CCNA

Purpose

This lab was an introduction to LLM's especially in the context of local LLM's for use in scenarios needing secure data protections. Becoming more

prevalent with every passing day, the want for LLM's, especially in hiring and portfolio management, has skyrocketed, and the need for data protection for the files being processed has similarly increased, requiring proprietary, or at least local LLM's as seen in this lab.

Background

Large Language Models, or more colloquially known as “AI,” are becoming much more prominent in almost all fields of society, especially as their accuracy, and consistency increases. Officially starting in 2018 with the first application of generative pre-training to Transformation Architecture in GPT-1, built as a research model by OpenAI, the following years were signs of a growing market in AI. Just two years later the first commercially available LLM, GPT-3, was released as part of an API, enforcing LLM's place in various industries. Recently the LLM market has boomed, with hundreds of different models becoming available for personal, public, and commercial use. Everything from Chat GPT's ever growing models, Grok AI from xAI, and its infamous racism issues, and the revolutionary DeepSeek AI, from the company with the same name.

AI as a concept has had long history, officially starting during the 1956, Dartmouth College, “thinking machines” summer conference, it has gone through many periods since. One of the first major successes, ELIZA, is seen as the first working chatbot ever to exist. Designed to emulate therapy by repurposing answers to questions previously asked, it was supposed to demonstrate the simplicity of AI at the time, but failed rather spectacularly, considering that the scientist who designed the AI found it hard to convince the subjects of ELIZA's inhumanity. In 1996, the computer system Deep Blue, played chess against the then world champion Gary Kasparov in a six-game matchup, losing five out of six games. Just a year later, Deep Blue won the rematch, and finished the last game in just 19 moves. In 2000, the robot Kismet proved capable of identifying and simulating human emotions, and in 2011, the first machine intelligences capable of recognizing human speech, albeit poorly were made by Apple and Amazon, Siri and Alexa respectively.

LLM's, are generally characterized as a predictive algorithm, and although this is rather simplistic, it is generally close enough to the truth for most people to get by, but as time goes on this becomes less and less true. Most of the LLM's we have today, definitely started out as predictive algorithms, weighing out how likely

one choice of character is to be preferred than the other, but now they are also trained through various other systems. Some of the systems that we have to train LLM's today have been around for a while and were used to train neural networks long before GPT-3 came into being, but they are often rather simple to understand. One of the most common ways is RLHF, or Reinforcement Learning from Human feedback, or to put it simply, the LLM will provide a variety of answers to a question, and then a human will go through them and rate them based on the quality of the response. If they did well the LLM gets what is essentially a reward, but if they did poorly, they get a punishment. Now these rewards and punishments are nothing more than increasing or decreasing an integer in the simplest form, but they are based off of the ways that we train other things, like pets. If the dog does a trick, and does it well they get a treat, but if they do it poorly, or not at all they don't get a treat, or even get a punishment. Other methods do exist, but this is one of the simplest, and one of those used most commonly.

What you'll be setting up in this lab is known as a Local Language Model, same acronym, different meaning. The main difference is that it runs on your local hardware and doesn't have access to the internet unless you specifically code an interface for it. This is incredibly important if you are going to be working with private or secure information through an LM as it isn't going to be scraping this information for the company that owns it. Putting information that is supposed to be confidential into an online LLM is risky, not only because they use that for training data, but also because it gives the company that owns that LLM access to that information, which Local LM's bypass entirely. This local exclusivity also means that you can customize it quite heavily using different interfaces coded separately.

Procedure

First install Ollama, by going to [Ollama.com](https://ollama.com) and selecting Download as highlighted below.

What will you build?

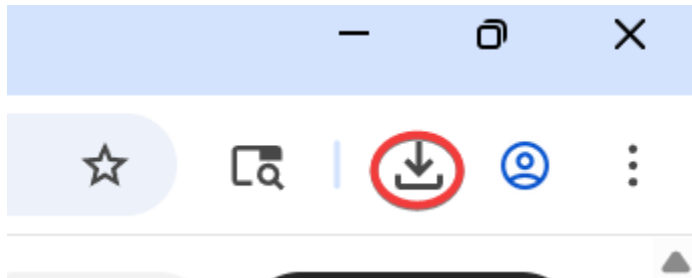
Ollama is the easiest way to automate your work using open models, while keeping your data safe.

Download

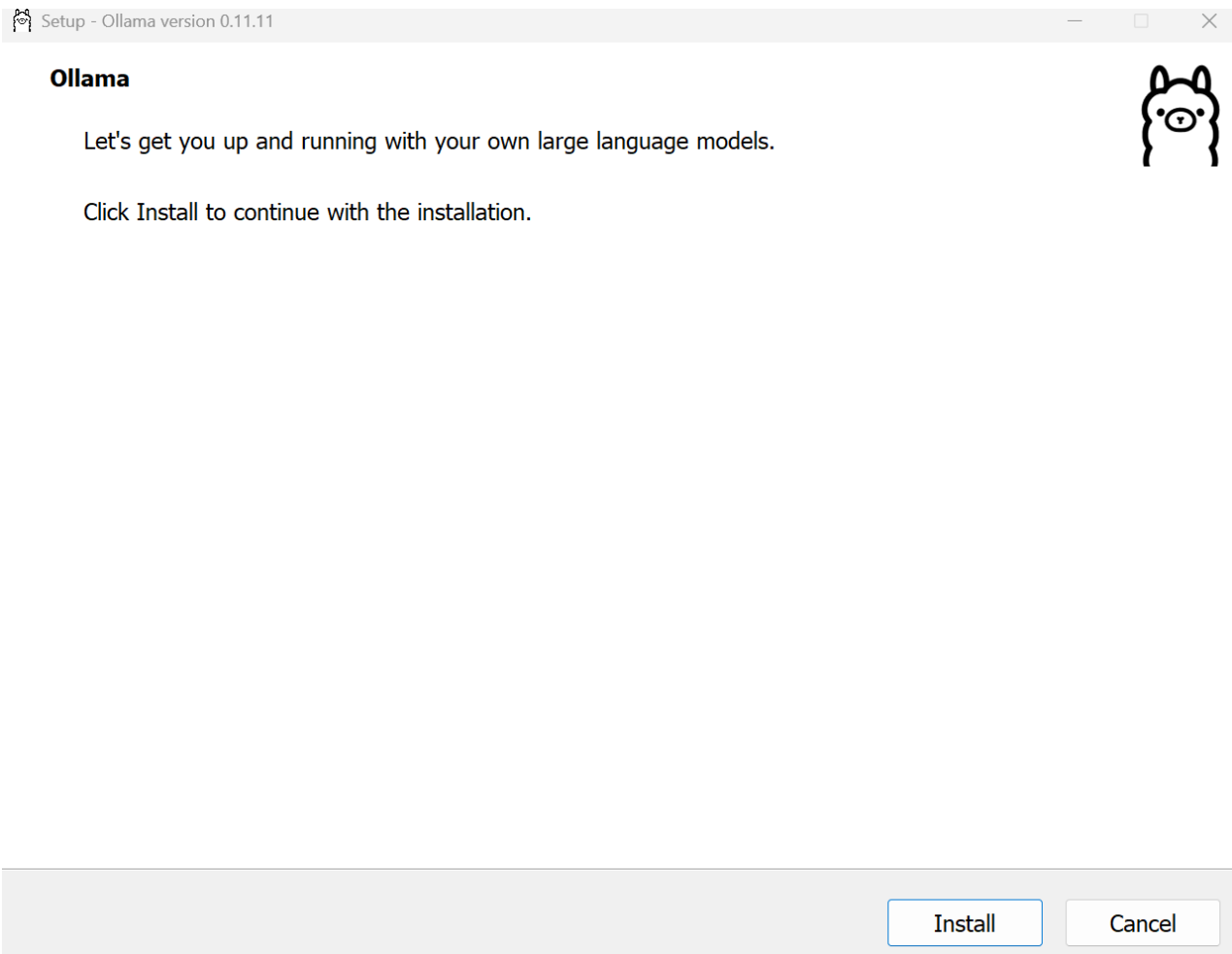


This should send you to this screen, where the Windows option is automatically selected. Click download

Go to the top right of the screen and select the downloads button, wait for the download to finish, and select the file OllamaSetup.exe that should have appeared



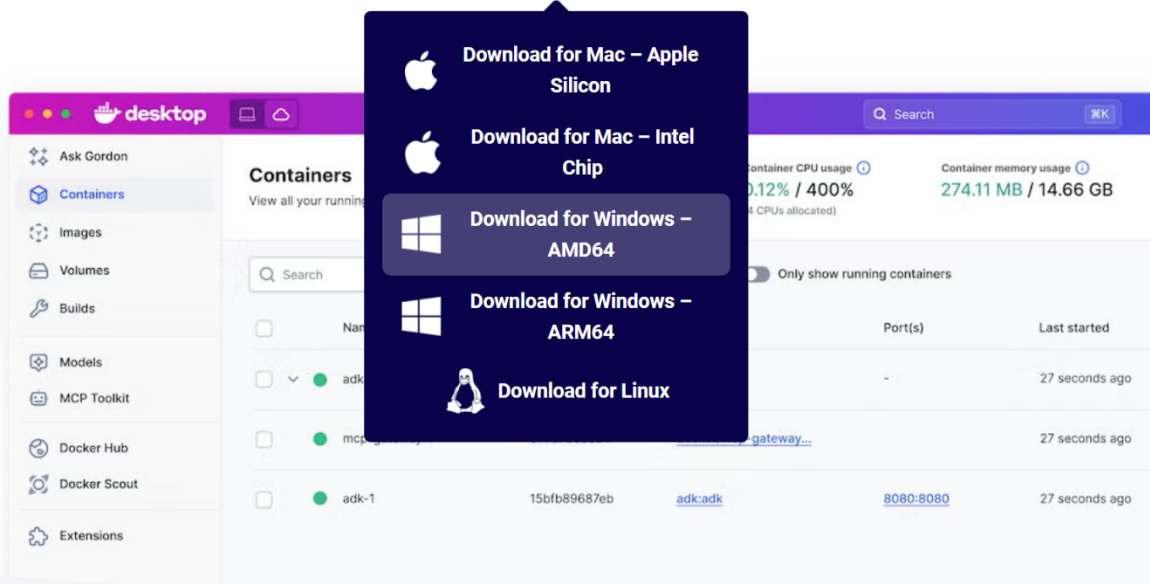
This should send you to the following screen, where you should select install and go through the installation process.



Next install Docker by going to docker.com and hover over the Download Docker Desktop button and select [Download for Windows – AMD64]

Develop Apps!

Your foundation for secure, intelligent development

[Download Docker Desktop](#)[Learn about Docker for AI](#)

Repeat the steps that you did for Ollama to find the download, and run the program before selecting the following options

Configuration

- ☒ Use WSL 2 instead of Hyper-V (recommended)
- ☐ Allow Windows Containers to be used with this installation
- ☒ Add shortcut to desktop

Windows Containers should only be enabled if you understand the risks. For more information, see [our docs](#).

OK

Restart if needed

Docker Desktop 4.46.0

Installation succeeded

You must restart Windows to complete installation.

Close and restart

When you restart this should pop up, follow the instructions and install Linux

```
C:\WINDOWS\system32\wsl.e  ×  +  v
Windows Subsystem for Linux must be updated to the latest version to proceed. You can update by running 'wsl.exe --update'.
For more information please visit https://aka.ms/wslinstall

Press any key to install Windows Subsystem for Linux.
Press CTRL-C or close this window to cancel.
This prompt will time out in 60 seconds.
```


Create your account if you don't already have one

Welcome to Docker

[Skip](#)

Work

Personal

Email address

Continue

Or





Create an account 



Sign in

Using Docker for work? We recommend signing in with your work email address.

Continue

OR



Continue with Google



Continue with GitHub



Continue with SSO

Sign into said account

Open Docker Desktop?

https://auth.docker.com wants to open this application.

☐ Always allow auth.docker.com to open links of this type in the associated app

Open Docker Desktop

Cancel

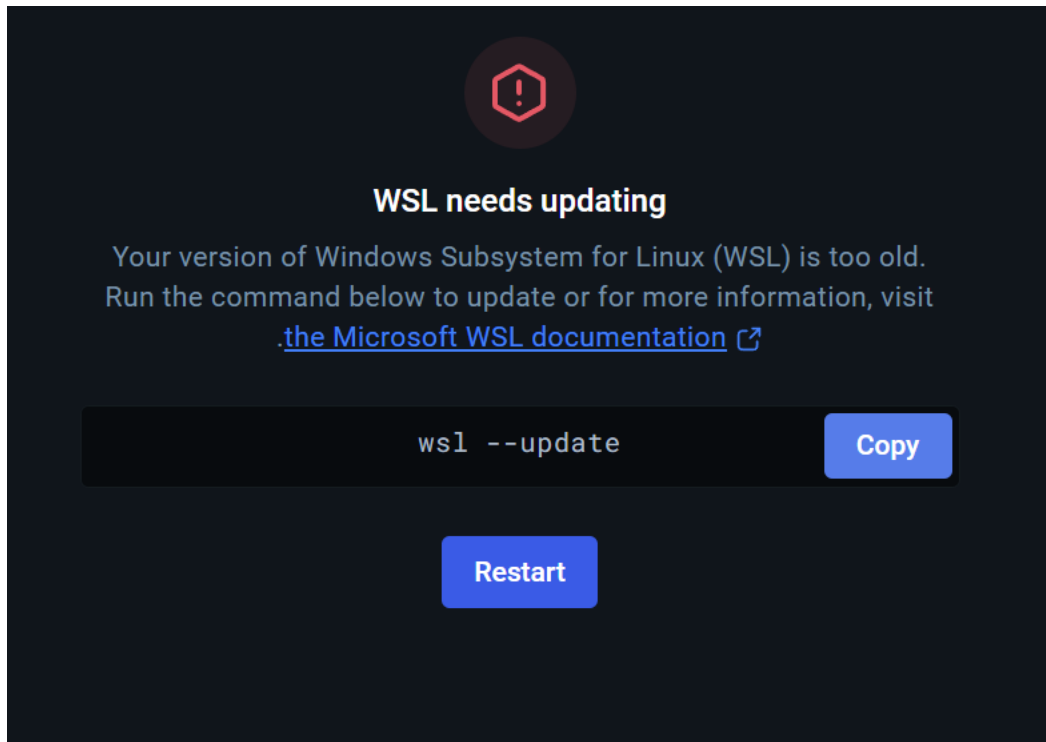


You're almost done!

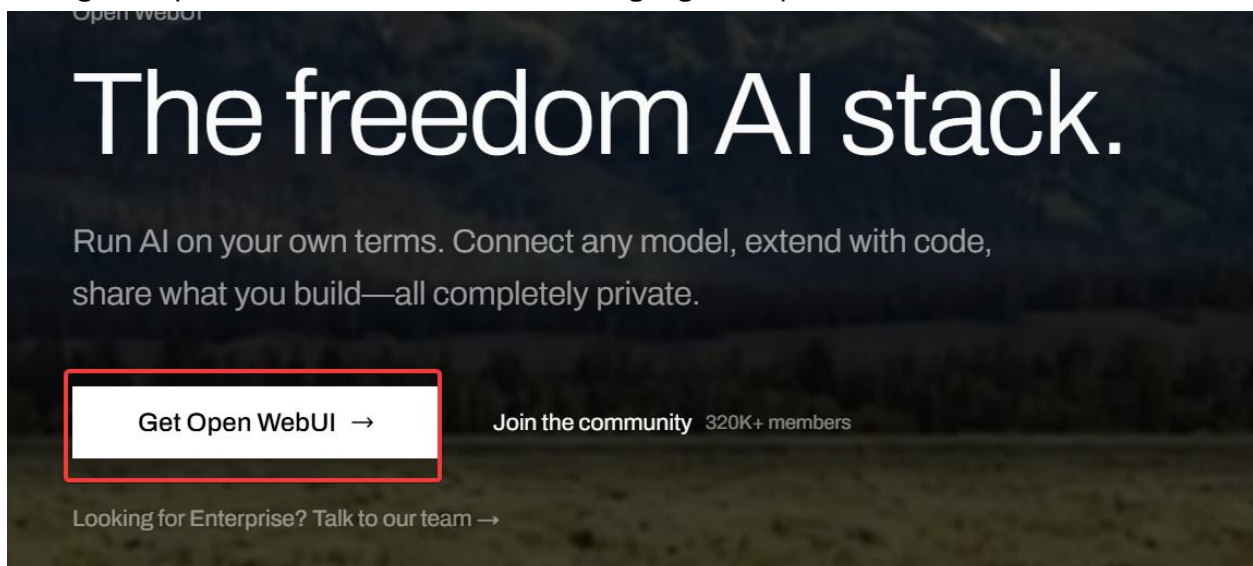
We're redirecting you to the desktop app.
If you don't see a dialog, click the button below.

Proceed to Docker Desktop

Then return to Docker Desktop and follow the recommendations for updates by pasting the provided command into the command prompt, before pressing restart.



Next go to OpenWeb UI.com and select the highlighted option below.



This should send you to GitHub, where you should scroll down until you see this. Copy the command by pressing the highlighted button below

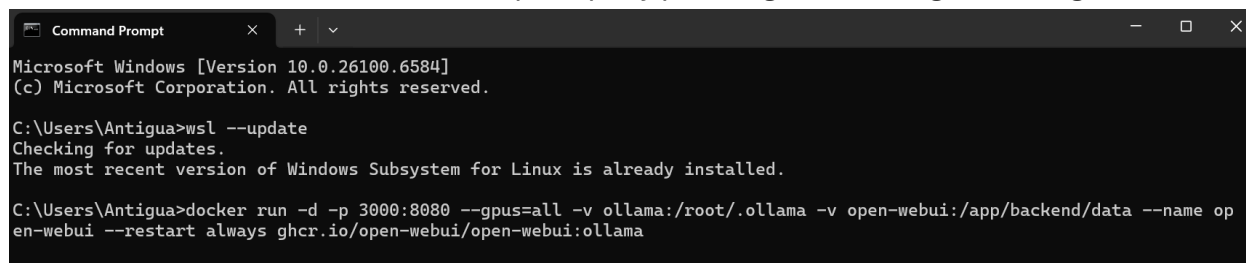
Installing Open WebUI with Bundled Ollama Support

This installation method uses a single container image that bundles Open WebUI with Ollama, allowing for a streamlined setup via a single command. Choose the appropriate command based on your hardware setup:

- **With GPU Support:** Utilize GPU resources by running the following command:

```
docker run -d -p 3000:8080 --gpus=all -v ollama:/root/.ollama -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:ollama
```

Then run the command in command prompt by pressing Ctrl V or Right clicking



```
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

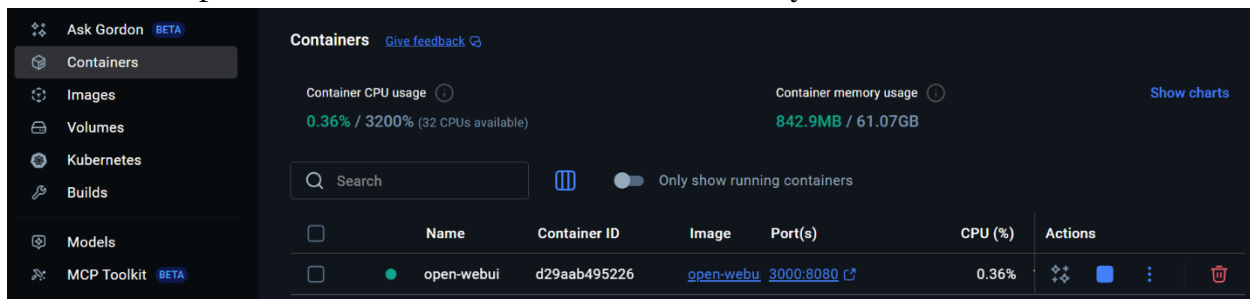
C:\Users\Antigua>wsl --update
Checking for updates.
The most recent version of Windows Subsystem for Linux is already installed.

C:\Users\Antigua>docker run -d -p 3000:8080 --gpus=all -v ollama:/root/.ollama -v open-webui:/app/backend/data --name op
en-webui --restart always ghcr.io/open-webui/open-webui:ollama
```

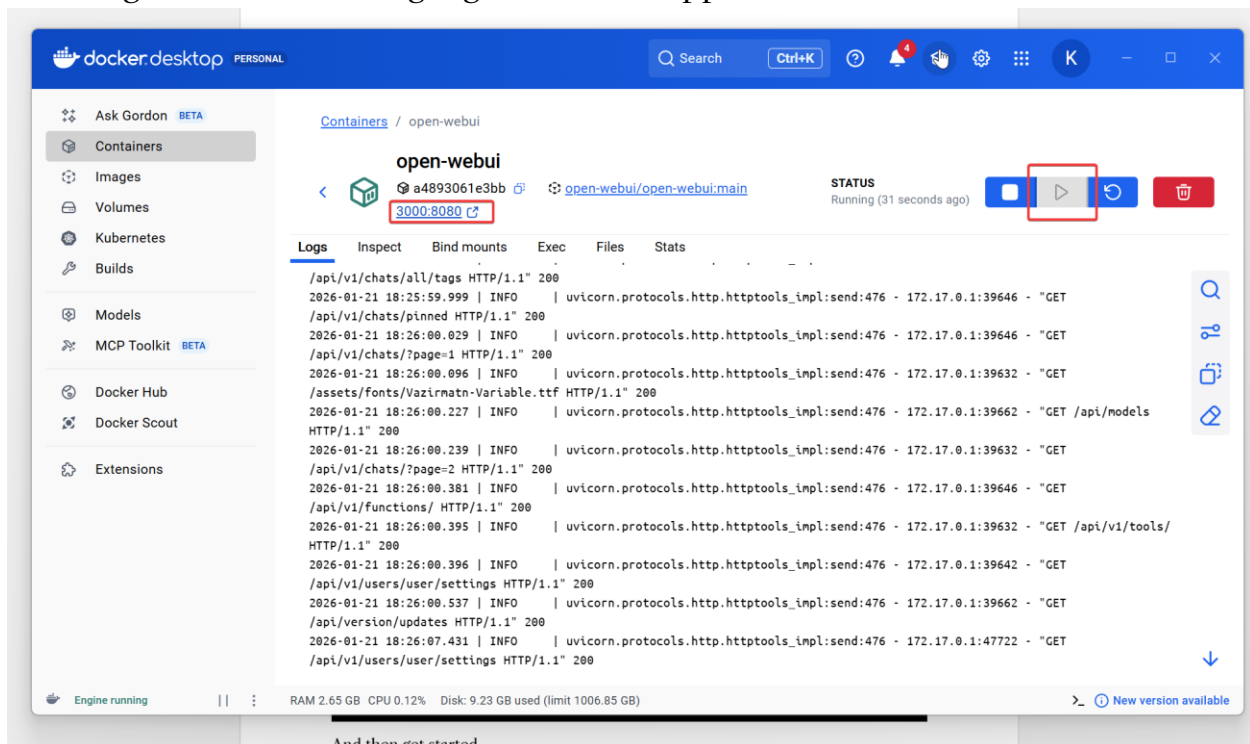
Wait for the pulls to complete

```
ollama: Pulling from open-webui/open-webui
d107e437f729: Pull complete
4f4fb700ef54: Pull complete
49b1c40da033: Pull complete
f1451649e055: Pull complete
427b6e68e4f6: Pull complete
45b71b5dc15d: Pull complete
ce4392d98604: Pull complete
dfcb19d2cc14: Pull complete
1fd1c0f46ae6: Pull complete
4079926b32d5: Pull complete
e07e6da0bc93: Pull complete
b0b6ac0f5373: Pull complete
a830e79592b2: Pull complete
f8043d35c69a: Pull complete
1ce95d9ae66c: Pull complete
659036234d55: Pull complete
```

Then start Openwebui in docker, if not automatically started

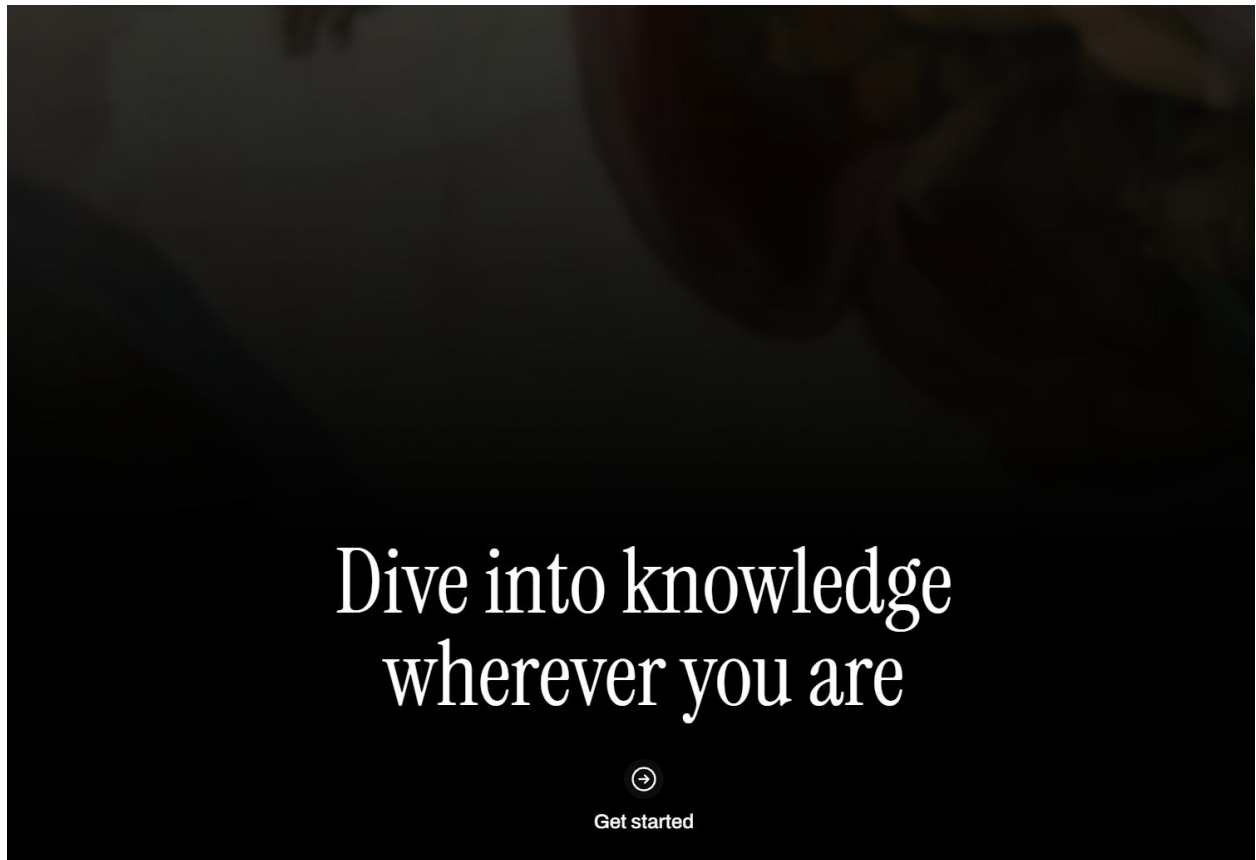


Run the program first by selecting the run button on the upper right side, then selecting the link shown highlighted on the upper left and wait for it to start.



And then get started

This should show up, and press [Get Started]



Then setup an account

Get started with Open WebUI

① Open WebUI does not make any external connections, and your data stays securely on your locally hosted server.

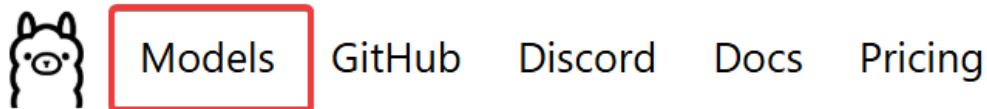
Name
Enter Your Full Name

Email
Enter Your Email

Password
Enter Your Password 

Create Admin Account

Next return to Ollama.com and open the Models tab located in the upper right of the screen



Choose a model to download like Llava

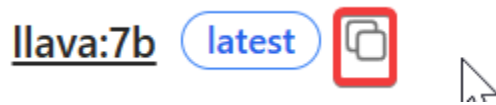
llava

 LLaVA is a novel end-to-end trained large multimodal model that combines a vision encoder and Vicuna for general-purpose visual and language understanding. Updated to version 1.6.

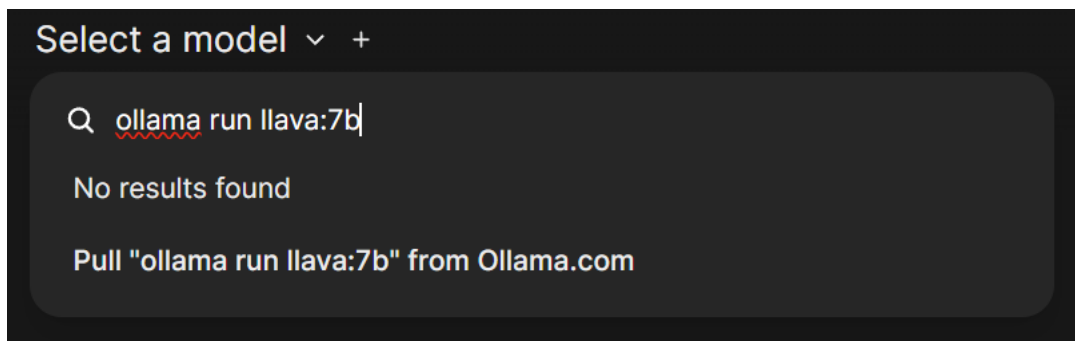
vision 7b 13b 34b

↓ 9.7M Pulls 🏷 98 Tags ⌚ Updated 1 year ago

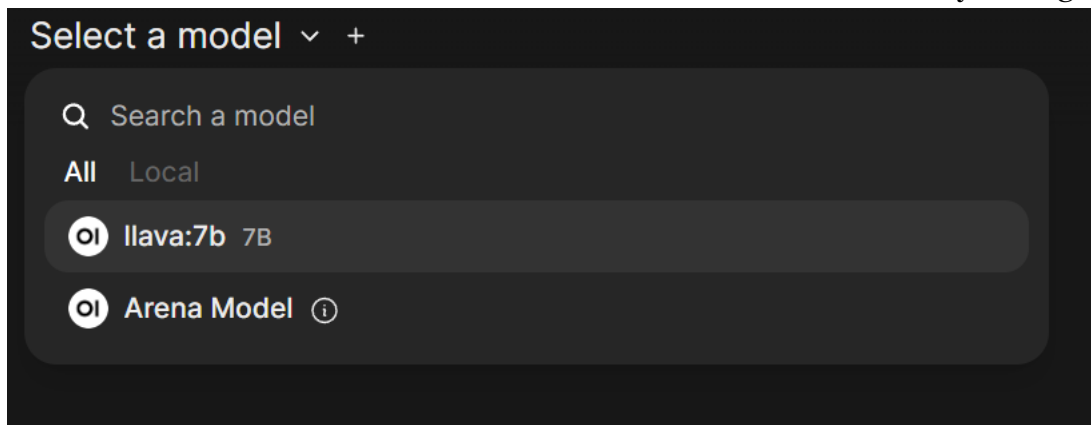
Select it and copy the command to the model that works best with your equipment by hovering over it and selecting the copy button that appears.



Then go to the localhost, also known as OpenWebui that you have running through docker, select the [Select A Model] tab in the upper left of the screen, paste in the command you got from Ollama, and press “Pull “model” from Ollama.com



Wait for the download to finish, and then select the model and you're good to go!



Extra

Because of the way that this works, as long as you have any device connected to the local network, you can access the model, although you will have to log into Openwebui. To do so, connect to the network, and input into the URL bar

[localhost: (first four digits found in the docker run)]

open-webui



 a4893061e3bb 

3000:8080 

And you should be able to connect to the models.

Problems

Although there weren't problems in the dozens, there were a couple of problems that I found. One of which was that when trying to access the models from another device through the localhost, I found that a regular user cannot access any of the models unless explicitly given permission by the admin account, which made it difficult to figure out if the models actually worked on another device. Not only that, Docker would often find itself in various states, and the localhost sometimes didn't load at all, but that was often either an issue with the program, or I needed to update the various programs involved.

Conclusion

This lab, as annoying as it was to figure out the requisite steps, was rather simple in its actual steps. The various uses for local AI, as tantalizing as they are, are also rather dampened by the lack of processing power that is going to define our future. The uprising of AI, and the continual buy in of the worlds richest seem to declare dark times ahead, especially because AI has never once made a profit despite what investors were promised. Despite that the actual experience of working with AI and docker especially will definitely be useful in the future, and although the duration was brief, every bit helps.

CCNP: Hosting a Local Language Model On A Network

Kazuyoshi Li-Hirata

